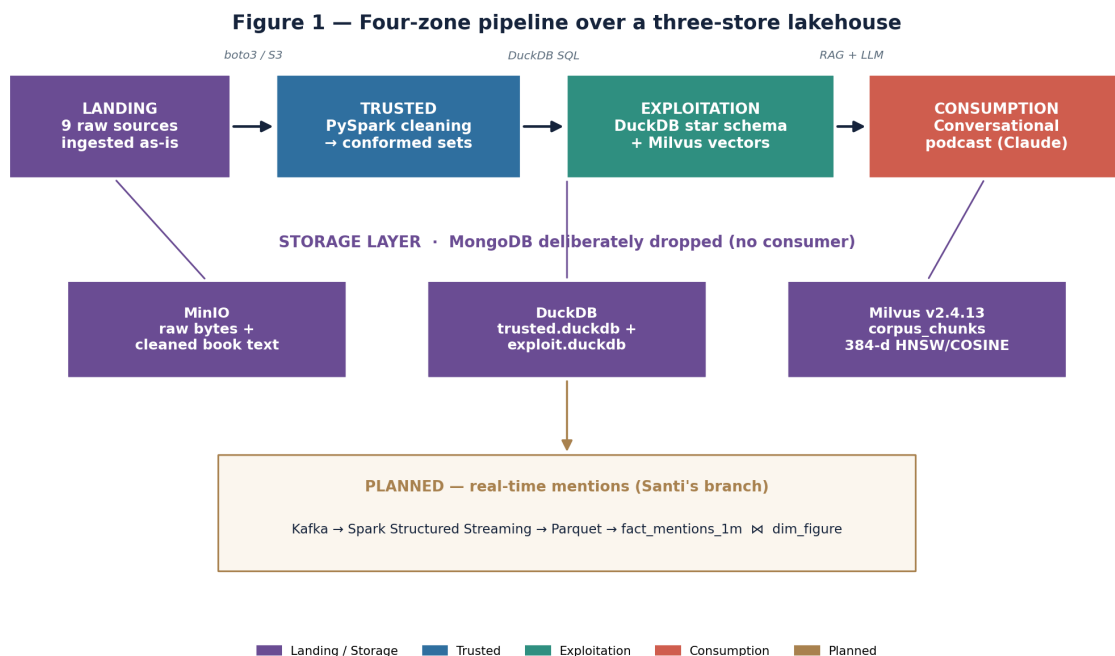


HISTORICAL CONVERSATIONAL AI

BDM · Project 2 — Technical Report

Exploitation & Consumption Zones — architecture, datasource usage, and design decisions



Author : Albert Puiggròs | Status : pipeline green end-to-end | Streaming : planned integration (Santi)

Contents

- 1 — Executive summary
- 2 — System architecture & the three-store decision
- 3 — Datasource inventory: what we use, partially use, and do not
- 4 — Trusted Zone (interface recap)
- 5 — Exploitation Zone: the star schema in full
- 6 — Consumption Zone: the conversational podcast
- 7 — Technology stack (named & versioned)
- 8 — Orchestration & runtime
- 9 — Design-decision register
- 10 — Planned streaming integration (Santi sync)
- 11 — Status & next steps

1 Executive summary

The deliverable is a **generated conversational podcast**. For any of nine historical figures — five philosophers (Plato, Descartes, Kant, Hegel, Nietzsche), two scientists (Einstein, Darwin) and two authors (Wilde, Twain) — the system produces a host-interview episode in which the figure answers *in character*, grounded in what they actually wrote, in a voice blended from their own aphorisms and prose, on an intellectually substantial theme loosely inspired by current news.

The engineering thesis is **consumer-justified design**: every store, table and tool exists because a named downstream consumer needs it. That principle is what removed MongoDB, what confined Spark to a single job, and what shapes the star schema. This report states each decision explicitly (§9), accounts for every datasource including the ones we do not use (§3), and names every tool with its version (§7).

Why this matters for grading

The course rewards architectural coherence over tool count. A reviewer can trace any byte from a raw source (§3) through a conformed Trusted set (§4), into a purpose-built Exploitation table (§5), to the exact agent that consumes it (§6). Nothing in the architecture is decorative.

2 System architecture & the three-store decision

The pipeline has four zones — Landing, Trusted, Exploitation, Consumption — over a **three-store lakehouse**: MinIO for raw and cleaned unstructured bytes, DuckDB for all tabular data (Trusted and Exploitation), and Milvus for sentence-embedding vectors. Each store has a non-overlapping role and at least one concrete consumer.

Figure 1 — Four-zone pipeline over a three-store lakehouse

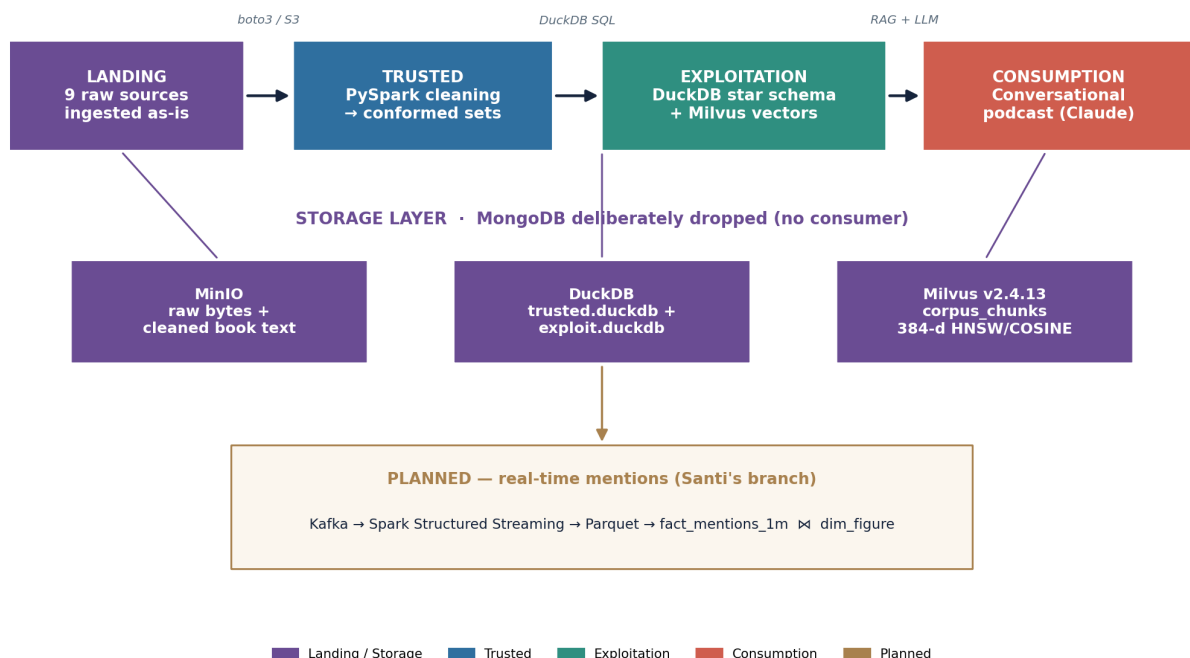


Figure 1 — Zones, stores, and the planned streaming arm. Edge labels name the mechanism that moves data across each boundary.

The MongoDB decision. The submitted design proposed MongoDB for figure profiles and the Wikipedia/Wikiquote/Gutenberg catalogs. After the Trusted Zone proved DuckDB flattens those

semi-structured sources cleanly, MongoDB had *no consumer left to justify it* — every reader of that data is a SQL join or a vector search. It was dropped. *figure_profiles* became a denormalised DuckDB dimension (*dim_figure*), not a Mongo collection.

3 Datasource inventory — used, partially used, unused

Nine datasources were ingested. Honesty about which ones the product actually consumes is itself a design statement: we keep partially-used and unused sources visible rather than hiding them, and we never pretend an unrelated source feeds the product.

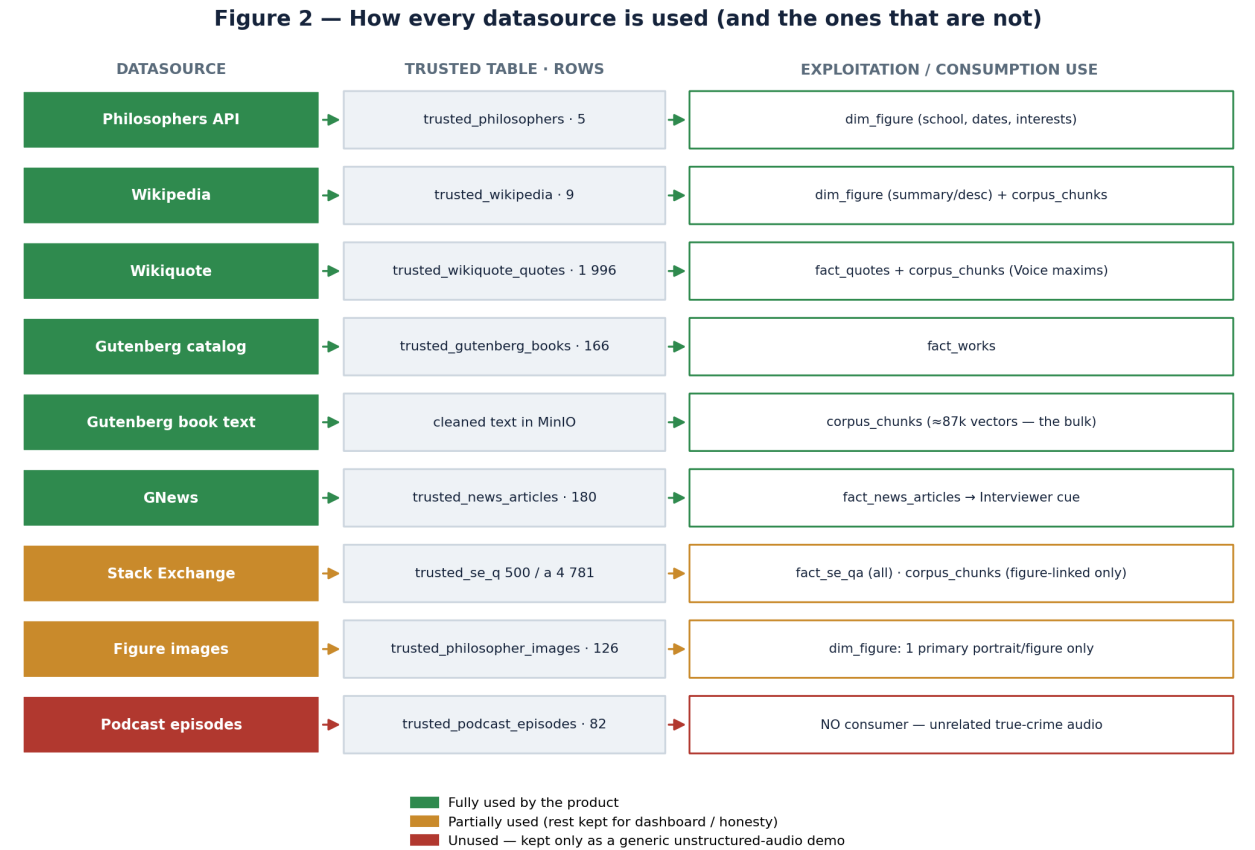


Figure 2 — Every datasource, its conformed Trusted table with live row counts, and exactly where it is consumed.

3.1 · The honest edges

- **Stack Exchange (partial).** All 500 questions with their best answer inlined populate *fact_se_qa* for the dashboard, but *only figure-linked* questions are embedded into Milvus — a question about “modal logic” with no figure tag has no place in a figure's RAG context, so it is deliberately excluded.
- **Figure images (partial).** 126 portrait records were ingested; *dim_figure* keeps only the single primary portrait key per figure (for a future dashboard/UI). The remaining images and the image bytes have no podcast consumer and are intentionally left unconsumed.
- **Podcast episodes (unused).** *trusted_podcast_episodes* (82 rows) is true-crime / entertainment audio metadata (american_homicide, betrayal_season_5) with zero connection to the nine figures and no product consumer.

Unused source — stated, not disguised

trusted_podcast_episodes does **not** feed the conversational podcast and we do not claim it does. It is retained solely as a generic *unstructured-audio handling* demonstration in the Trusted Zone. Calling this out is the correct engineering posture and the expected report rationale.

4 Trusted Zone — the interface this report builds on

The Trusted Zone is upstream of this report's focus, so only its contract matters here. Nine heterogeneous sources are ingested to MinIO and cleaned by **PySpark** jobs into conformed, deduplicated, type-safe sets. Spark is justified precisely here: it reads JSON and book text from MinIO in parallel at volume. Outputs land as *trusted_** tables in *trusted.duckdb* plus boilerplate-stripped Gutenberg text back in MinIO. These conformed sets — and nothing else — are the Exploitation Zone's inputs.

5 Exploitation Zone — the star schema in full

The Exploitation Zone reshapes the conformed sets into a **star schema** plus a **vector corpus**. A star schema is the right model because every analytical and conversational question is “something *about a figure*”: one conformed dimension, several fact tables radiating from it, no snowflaking. All tabular work is pure DuckDB SQL — source and target are both DuckDB and the joins span <50 dimension rows, so a Spark JVM would add startup cost with zero throughput gain. Spark is used in exactly one Exploitation job: embedding (\$5.4).

Figure 3 — Exploitation star schema (grain · keys · columns)

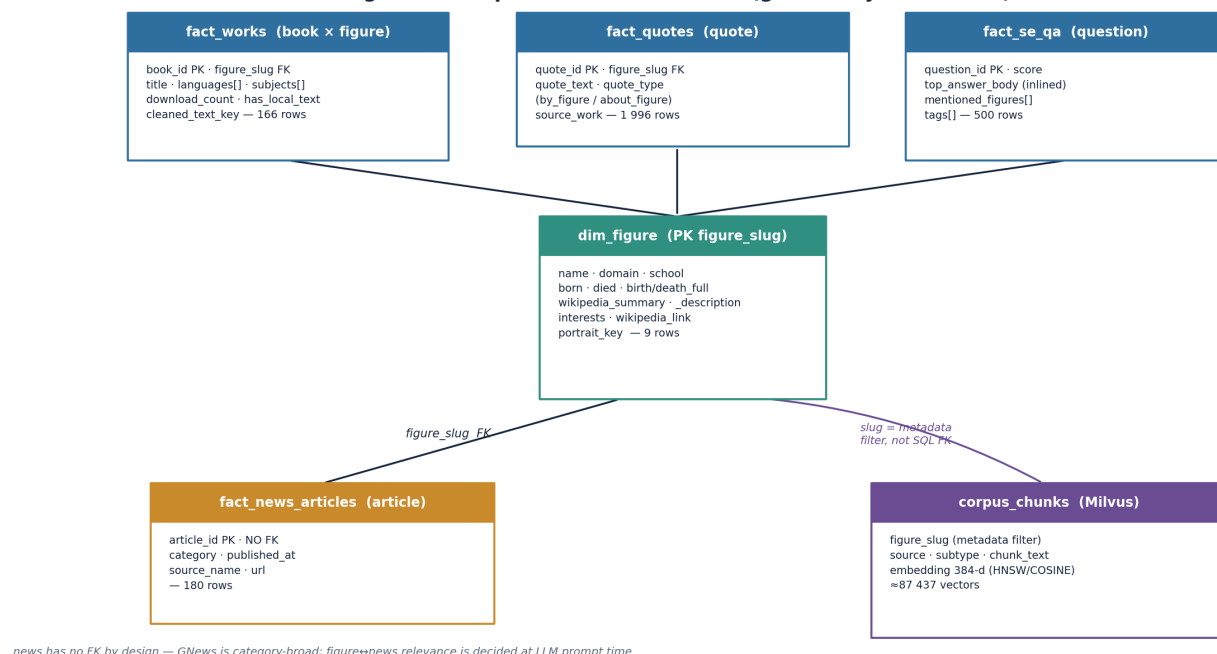


Figure 3 — The conformed dimension, four facts, and the Milvus corpus, with grain, keys and representative columns.

5.1 · dim_figure — the conformed dimension

One row per figure (9 total). It anchors on the nine-figure character registry and LEFT JOINs the Trusted sets, so the five philosophers carry school / signed-year birth-death / full dates / interests while scientists and authors honestly carry NULL there. It also conforms the Wikipedia one-line *description* and first-paragraph *summary* — the factual anchor every agent reads — and a single primary portrait key.

5.2 · The fact tables and their derivation

Fact	Grain	Derivation logic (the interesting part)
fact_works	book × figure	INNER JOIN dim_figure; <i>all</i> books kept, has_local_text flags the embeddable subset so the KPI counts authored works, not just embeddable ones
fact_quotes	quote	quote_type splits by_figure (authentic voice → Voice agent) from about_figure (third-party → Reasoner); both retained so each consumer filters
fact_news_articles	article	Pass-through with no figure_slug : GNews pulls by broad category, so figure relevance is decided later, by the Interviewer at prompt time
fact_se_qa	question	Top answer inlined (accepted > highest score) to save a join; mentioned_figures[] derived via tag-prefix + word-boundary regex on title/body (e.g. cartesian→descartes)

5.3 · Why news has no foreign key

This is a deliberate modelling decision, not an omission. Attaching a figure_slug to a world/science/technology headline would fabricate a relationship the data does not contain. Instead the Interviewer agent reasons figure↔news relevance at LLM time, given the recent-news set and the identity card — relevance is inferred where the reasoning lives, not frozen incorrectly in a column.

5.4 · corpus_chunks — the Spark scalability proof

The embedding job is the project's deliberate scalability demonstration. It reads ~166 books from MinIO, windows each into **220-word chunks with 40-word overlap**, embeds them with **all-MiniLM-L6-v2** (384-d, L2-normalised for COSINE) and writes them to the Milvus *corpus_chunks* collection — HNSW index (M=16, efConstruction=200), chunk text inlined so retrieval is a single call. Small sources (Wikipedia, Wikiquote, figure-linked SE) are added driver-side. The verified collection holds **≈87,437 vectors**.

The scaling lesson is the execution model. Spark parallelises *only* the embarrassingly-parallel MinIO read+chunk; embedding and insertion then **stream to the driver** via toLocalIterator() in 1,000-row micro-batches with one model load — memory is constant regardless of corpus size. The naive alternative (collect() everything, load the model per partition) is what fails to scale; demonstrating that contrast is the point of using Spark here at all.

6 Consumption Zone — the conversational podcast

Four cooperating agents turn the Exploitation stores into a single-figure interview. Generation is injected as a provider-swappable *llm_fn* (Anthropic Claude); with no API key the pipeline still emits the full episode structure, so it is testable offline. The governing principle is **strict separation of facts and style**.

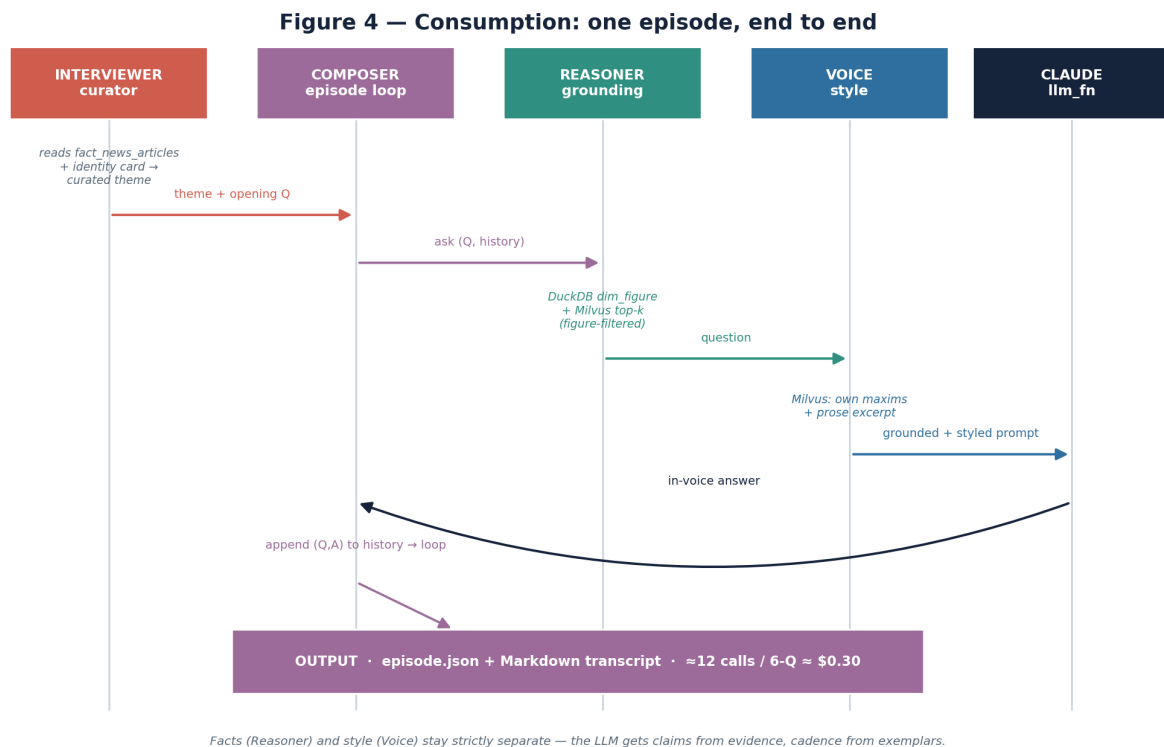


Figure 4 — One episode: who calls whom, which store each agent touches, and the conversational loop.

6.1 · Reasoner — substance

Loads the DuckDB identity card and retrieves the top-k figure-filtered passages from Milvus, then assembles one grounded prompt that forbids assertion beyond the evidence and demands in-line citation of the retrieved passages. If Milvus is unreachable it degrades to identity-card-only rather than failing — the podcast never crashes, it gets less specific.

6.2 · Voice — blended style

Reuses the Reasoner's retrieval twice: the figure's own Wikiquote maxims (*source=wikiquote, subtype=by_figure*) for cadence, and one Gutenberg prose excerpt for register — both queried with the interview question so the exemplars are topically adjacent. The result is injected as a style block that changes how the answer *sounds*, never what it asserts.

6.3 · Interviewer — a topic curator (not a news reader)

The Interviewer curates evergreen, intellectually rich themes (the nature of AI, scientific truth, social behaviour, ethics) and uses headlines only as loose inspiration when they genuinely fit the figure. It opens with a curated question, then generates each follow-up from the transcript so far — a real adaptive interview, not a fixed questionnaire.

Content-safety hazard — and the structural fix

RAG over a complete historical corpus **will** surface period bigotry if charged news is matched against it (an observed failure: a Gaza headline pulled an antisemitic passage from Kant's *Anthropology*). The mitigation is structural, not a post-hoc filter: the Interviewer is a curator that steers to intellectual themes and avoids raw geopolitics, and the cold-open quote keys off the curated *theme*, never the raw headline — so the failure mode is designed out.

6.4 · Bio-grounded delivery (this iteration's work)

The identity card now derives, from *curated data only*, a *voice_descriptor*: full birth/death dates when ingested, an exact lifespan via signed-year arithmetic (Plato $-428 \rightarrow -348 = 80$ years), and the Wikipedia one-line description — which already encodes origin and role (“German-born theoretical physicist”). No native language or accent is invented; the model is instructed to let period and stated origin colour the register, nothing more. The same string is the intended seed for the future text-to-speech voice.

6.5 • Episode composer

Stitches a theme-keyed cold-open quote, the host framing, the conversational turn loop (prior Q/A threaded into every answer for coherence and non-repetition) and a sign-off into *episode.json* and a readable Markdown transcript. A six-question episode is ≈ 12 Claude calls, $\approx \$0.30$ on Sonnet.

7 Technology stack — named and versioned

Every tool below has a justifying consumer; nothing is present for its own sake.

Layer	Tool	Version	Role / justification
Orchestration	Apache Airflow	2.9.0	Two DAGs, LocalExecutor; trusted auto-triggers exploitation
Airflow metadata	PostgreSQL	13	Airflow's own backend DB
Batch processing	Apache Spark (PySpark)	3.5.x	Trusted cleaning + the one embedding job (Java 17 JRE)
Tabular store	DuckDB	1.x	trusted.duckdb + exploit.duckdb; single-writer
Object store	MinIO + mc	latest	Raw bytes + cleaned book text; S3 API via boto3
Vector store	Milvus	v2.4.13	corpus_chunks; standalone + etcd v3.5.5 + internal MinIO
Milvus client	pymilvus	2.4.x	Pinned to the v2.4 server
Embeddings	sentence-transformers	≥ 3.0	all-MiniLM-L6-v2, 384-d (CPU torch 2.x)
LLM	Anthropic Claude	SDK ≥ 0.40	claude-sonnet-4-6 default; provider-swappable llm_fn
Streaming (planned)	Kafka + Spark SS	cp-kafka 7.5.0	character-mentions topic $\rightarrow$ fact_mentions_1m
Interchange	PyArrow / Parquet	≥ 15	Streaming output; deltalake present, not yet used
Report tooling	matplotlib + reportlab	3.10 / 4.5	This document (pdf skill path)

8 Orchestration & runtime

Two Airflow DAGs run with the LocalExecutor and manual trigger. *bdm_p2_trusted_zone* cleans the nine sources and, on completion, auto-triggers *bdm_p2_exploitation_zone* via TriggerDagRunOperator. The Exploitation DAG gates on MinIO and Milvus health, builds *dim_figure*, then the facts.

Why fact builds are serialised

DuckDB is single-writer: concurrent writers collide on the file lock. The four fact builds are therefore chained with Airflow's *chain()* rather than parallelised — correctness over a few seconds of wall-clock. *corpus_chunks* then embeds, and two verifier tasks (DuckDB row sanity, per-source Milvus existence) gate completion.

Milvus runs as a standalone Docker stack (server + etcd + an internal-only MinIO kept portless so it cannot collide with the lakehouse MinIO). The embedding dependencies and the Anthropic SDK are baked into the Airflow image; the consumption package is mounted in so it has the dependencies and can reach Milvus.

9 Design-decision register

The decisions a reviewer should be able to interrogate, each with its forcing reason.

#	Decision	Forcing reason
D1	Drop MongoDB	DuckDB flattens the semi-structured sources; Mongo had no consumer left to justify it
D2	Spark in exactly one Exploitation job	Tool-justification rule; SQL beats a JVM for <50-row joins
D3	Star schema, dim_figure conformed	Every question is “about a figure”; avoids snowflaking
D4	news has no figure FK	GNews is category-broad; a forced FK would fabricate relationships
D5	Inline top SE answer	A question without its answer is half the signal; saves a join
D6	Embed only figure-linked SE	Non-figure Q&A has no place in a figure's RAG context
D7	Stream-to-driver embedding	Constant memory regardless of corpus size; the scalability point
D8	Facts vs style separated	Claims from evidence, cadence from exemplars — no leakage
D9	Interviewer = curator	Structurally defuses corpus-bias on charged news
D10	voice_descriptor from curated data only	No invented native language; honest seed for TTS
D11	Serialise fact builds	DuckDB single-writer file lock
D12	Streaming stays Parquet, integrates by rebase	Santi's branch forked pre-Trusted; fast-forward would delete our work

10 Planned streaming integration (Santi sync)

Santi's branch runs a Spark Structured Streaming job: it reads the Kafka *character-mentions* topic, applies a 1-minute tumbling window with a 30-second watermark, aggregates to (window_start, window_end, character_name, domain, mention_count, avg_sentiment) and writes Parquet to */app/trusted/streaming/fact_mentions_1m*. It slots into the Exploitation Zone as a real-time fact joining *dim_figure*.

Figure 5 — Planned streaming integration & open decisions

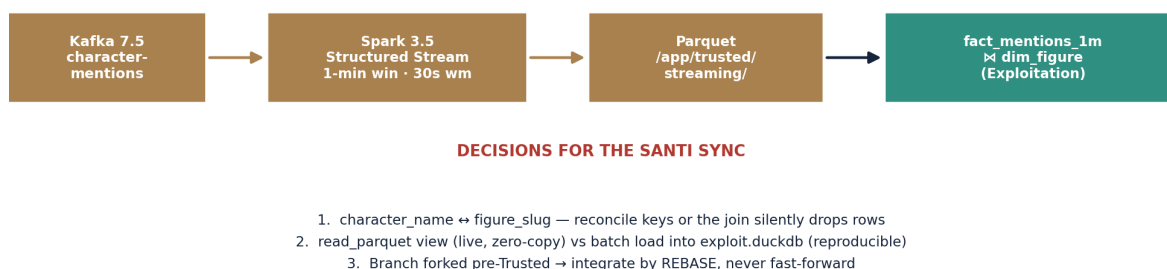


Figure 5 — The streaming seam and the three decisions to settle in the sync.

- **Naming.** *character_name* must reconcile with our *figure_slug* key, or the join silently drops every row.
- **Landing form.** A DuckDB *read_parquet* view (always fresh, zero-copy) versus a periodic batch load into *exploit.duckdb* (reproducible offline). Recommendation: a live view for the KPI, materialised on DAG run for reproducibility.
- **Merge.** origin/santi forked *before* our Trusted commits, so its diff shows our work as deleted — integration is a **rebase**, never a fast-forward.

Once joined, *fact_mentions_1m* gives the dashboard a live “who is being discussed now” signal and can feed the Interviewer a real-time topical cue — closing the loop between the streaming and consumption ends of the pipeline.

11 Status & next steps

Component	Consumer	Status
<i>dim_figure</i> + 4 facts	Dashboard KPIs, agents	Green
<i>corpus_chunks</i> (Milvus)	Reasoner / Voice RAG	Green · ~87,437
Conversational podcast	End user (demo)	Green · live
Bio voice_descriptor	Text register + future TTS	Done this iteration
Streamlit dashboard	Project demo	Planned
<i>fact_mentions_1m</i>	Real-time KPI	Planned (Santi)
Text-to-speech	Audio episode	Planned

The pipeline is green end-to-end: the Trusted DAG auto-triggers the Exploitation DAG, both succeed, the vector corpus verifies, and live episodes generate. The immediate next steps are the Streamlit dashboard, the streaming integration with Santi (§10), and the text-to-speech path seeded by the bio descriptor (§6.4).